

Lec 23 Reparameterization Trick

Lec 23 Reparameterization Trick

Lecture	23 / 63 + intro
Duration	35:04
Captions	English (auto-generated)
Words	5,422
Video	GA9T9kNoRd4
Playlist	Fundamentals of Generative AI and LLMs

Auto-generated YouTube captions, lightly cleaned (removed [music] tags, grouped into paragraphs). These are **verbatim lecture captions**, not edited notes.

Transcript

[00:00] Hello everyone. Welcome to the lecture series on fundamentals of generative AI large language models. We are in the week three and in this particular session, we will be looking into one of the most important concept called re-parameterization trick. This is very important and then this will give you the complete intuition why so far we have never seen the architecture of VAE. Towards the end of this particular session, you will see the architectural diagram of VAE. Although we have seen the encoder decoder, then we have seen the loss function, but we will be looking in detail and by end of this you will be getting an architectural diagram. So, this we have already seen in our previous session that this is the

[00:45] loss function of VAE which has totally two components. One is we have an reconstruction term and the second is regularization term. What exactly is a reconstruction? Now, if at all if we have an continuous data, then this particular first term will be taking a mean squared error. Suppose if we are taking a binary data, then the first term will be a binary cross entropy. So, basically this particular term will tell us how far the reconstructed data will be similar to the original input. So, that is what is the first term. Second term what we have is the regularization. What is

regularization here? It's a KL divergence. Now, KL divergence since both of these are Gaussian distribution in a closed form,

[01:42] the KL divergence is written in this particular form. So, now what exactly KL divergence this particular regularization term ensures that the encoded distribution remain a to the prior distribution. Now, what exactly is the problem here? Now, when you look into the entire loss function, we have somewhere is Z . Now, Z is sampled from this particular distribution. What is the sampling here? We are randomly picking one value from this particular distribution with the parameters mean and variance.

[02:20] Now, when Z is a randomly sampled, then there involved a random operation. So, since sampling is a random operation, always randomness breaks the gradient flow. If there is no gradient flow, then how do we compute the back propagation? How do we update the parameters? Therefore, it makes the back propagation impossible. Now, the question is the solution for this particular one is reparameterization trick. And what is the question? How do we compute the gradients when part of the network involves a sampling? Therefore, we require this particular concept called reparameterization trick.

[03:02] So, let us get into the details. Now, we'll be going very slowly. We will be looking into each step. So, towards the end of this particular session, you'll have a complete clarity. Now, what is seen here? There is an input. When input is a passed to an encoder, what is that encoder will give us? Encoder output the parameters. What parameters? The parameters of an appropriate posterior distribution. Which means encoder is going to give us this mean and then the variance. Then, once we get this output, what is that we are going to do? What is the the next step? By now, you should be able to answer to this. So, the next step is Z is sampled from this particular distribution.

[03:51] How is that you'll be writing mathematically? Z is a sampled either from this or the Z is a sampled from the parameters from this a distribution. So, both are the same. So, once a Z is being a sampled, what is the next step? So, you'll be passing a Z to an a decoder. So, Z is which is a latent space Z is then passed to to an a decoder. Now, once we pass Z to the decoder, what is the role of a decoder? Decoder has to reconstruct. It has to give us the output. And that output we call it as a reconstructed one or we say that a decoder has a generated the data.

[04:30] Mathematically, how is that you will be writing it? So, $\hat{X}$ is the reconstructed data, right? And to that Z is a passed as an input. And [snorts] here f of θ here is an you know, the decoder neural network. Therefore, when we see Z is a latent vector and then f θ is the decoder neural network and the θ here represents the decoder parameters. These decoder parameters are weights and biases. Now, these are the learnable parameters. So, once you update these parameters, definitely $\hat{X}$, whatever is the reconstructed value, becomes closer to the original

input. Yes?

[05:19] Therefore, $\hat{X}$ is a reconstructed output which is produced by the decoder. What is that we are seeing in this particular slide? We are seeing the feedforward process. Input is passed to an encoder. Encoder is going to give us the distribution parameters. Then Z is sampled from this a distribution. Then Z is passed to to an a decoder. Decoder is a neural network which it takes Z as an input and produces the data, which is $\hat{X}$, denoted as $\hat{X}$.

[05:51] Now, whatever are the decoder parameters θ , these are the learnable parameters. So, once this is done, what should be our next step? The whole idea here is to see whether we can able to compute the back propagation. So, let us now see what exactly is going to happen. Now, when Z is passed as an input to an decoder and decoder if if input is an tabulated data, then we can imagine that encoder-decoder can be a simple multi-layer perceptron. So, now let us assume the decoder here is a multi-layer perceptron model. So, Z is a passed as an input to the decoder. Now, what is that the decoder has to give?

[06:43] And how is that the process is going to happen? First, you are going to when Z is passed to an first hidden layer, which we denote it as a H_1 . How is that we perform? Z is passed as an input. Therefore, Z gets multiplied with the weights. Let us assume those weights are denoted by W_1 . And you'll be adding the bias at term B_1 . And since it is a hidden layer, we will be also using an activation function. And that activation function here is a ReLU. So, that is what is being written here. So, what exactly operation has taken place?

[07:21] The decoder takes the latent vector Z , multiplies it with the input weight matrix W_1 , and then add the bias B_1 . And then what is that we are doing? We are applying the activation function, and here the activation function used is a ReLU. Therefore, H_1 is the first hidden representation inside the decoder. Okay. So, once you pass Z to the hidden one hidden layer, we're going to get the output. Now, that output will be passed to the next layer. So, here the next layer is the output layer. So, how is that you will be passing? What is that we are passing?

[08:03] Here, the output is denoted as a H_1 . This H_1 will get multiplied with that particular weight matrix and then you'll be adding the bias. And definitely, there will be an activation function there as well. So, is it not the equation is written in this way? So, H_1 gets multiplied with the weight matrix W_2 and then we are adding the bias. Now, that entire you'll be passing it to an activation function. So, what activation function what is the F here? That primarily depends on what kind of data we are reproducing, right? So, how is that you're doing? H_1 is the hidden representation that is a passed to the next layer. Which is the next layer here? It is an output layer. Primarily for what? For reconstructing the

[08:50] input. Now, here F is an activation function that primarily depends on what

type of data that we are trying to reproduce. So, here the possibility is either you can have a continuous data or you can have a binary data. Suppose if it is a binary data, we will be using a sigmoid activation function. If we are using a continuous data, then we'll be using a linear activation function. So, so far this is fine. Now, what are the learnable parameters here?

[09:21] Is it not a W_1 , W_2 , B_1 , and B_2 ? Therefore, at the decoder side, the parameters are W_1 , B_1 , W_2 , B_2 . Now, once we get the output, what is the next step? What is the next step? Can you recall? The next step is to compute the uh loss. So, once the output $\hat{X}$ is obtained, we will be computing the loss. Now, I'm writing very generalized representation here because that loss depends on either it can be a mean squared error or it can be a binary cross entropy. So, we will be computing the loss and the loss will be between $\hat{X}$ and X .

[10:02] Once we compute the loss, what will be the next step? You have to update the parameters. Why? Initially, the loss value will be high. Then, we will be updating the parameters. Then, over a period of time, the weights we are going to get the optimized and the bias will be an optimized one. So, once $\hat{X}$ is obtained, we have to compute the loss. And that loss will be between $\hat{X}$ and X . Then, we will be updating the parameters.

[10:31] Now, if at all if you want to update the parameters somewhere from our week one, go back and then recall. What is that we need to do? We have to compute the gradients, right? So, how is that we compute the gradients? So, first we compute the gradients with respect to W_2 and B_2 , right? So, how is that we can write this particular derivative if you want to find out? Is it not derivative of loss with respect to W using a chain rule?

[11:00] Is it not first you will be computing with respect to $\hat{X}$ and from $\hat{X}$ we can reach it to W_2 . So, is it not we write in this particular form using a chain rule? So, derivative of loss with respect to W_2 will be loss with respect to $\hat{X}$ and $\hat{X}$ with respect to W_2 . Similar way, even the bias term. So, derivative of loss with respect to B_2 will be derivative of loss with respect to $\hat{X}$ and $\hat{X}$ with respect to B_2 . So, this is with respect to two the output layer.

[11:37] So, from from here you can able to compute all of these values. There is no problem. Now, further if we go back, further going back is reaching now to hidden layer. So, at hidden layer what are the parameters to update? Is it not the W_1 and B_1 ? How is that we will be reaching using a chain rule? Is it not a derivative of loss with respect to W_1 and derivative of loss with respect to B_1 ? How is that to be computed? It is the derivative of loss with respect to $\hat{X}$ and from $\hat{X}$ we reach to H_1 and from H_1 we reach to W_1 . So, is it using a chain rule is it not this is the form equation? So, if you want to update the weight W_1 , the path during back propagation the path is first you will be reaching to $\hat{X}$ and

[12:33] from $\hat{X}$ you will be reaching to $H1$. From $H1$ you will be reaching to $W1$. So, this is what is the chain rule. So, here also we can compute values for each of the term. So, therefore can we say that the gradients with respect to the decoder parameters $W1, W2, B1, B2$ are all computed using a standard back propagation. Hence, in VAE or in a standard VAE decoder side is not a problem. So, this is what we have reached. So, till where we have computed the gradients? We have computed the gradients only till the decoder part.

[13:16] So, now let's proceed to the next step. So, I hope up till decoder it is clear. Now, what is the role of a decoder? Decoder has to take an input to Z and then it has to reconstruct the data or it has to produce the output. Which means whatever the output that the decoder is produced, is it not from Z only? So then the loss whatever we are getting definitely depends on Z through that exact, right? Are you getting this? Now if at all if you are computing the loss and if the decoder has to produce the output, that output has to come by taking a Z as an input. So which means the loss depends definitely on a Z as well. So now we are computing that particular using a chain rule. So what is that we have mentioned here? Since a

[14:09] decoder uh um output is a produced from a Z and this is the mathematical equation, then definitely loss depends on a Z through $\hat{X}$. How is that we will be computing the now this derivative derivative of loss with respect to Z ? Is it not using a chain rule? Derivative of loss with respect to Z is computed as loss with respect to $\hat{X}$ and from $\hat{X}$ we can reach it to Z . So this is how we have a computed the derivative. Now you can ask how we have a computing. Once you sample a Z from that particular distribution, is it not a Z is just a numerical numbers? Is it not a vector? So from [snorts] our session all the sessions we have understood. So Z is a sampled. Once you sample it is just one a numerical

[15:04] number. So definitely that is what is written here. Since a Z is a treated as a numerical input to the decoder, definitely the decoder is a differentiable with respect to its input. Therefore, we have reached successfully the gradients back from reconstruction till this Z . So, I hope uh this is uh clear. What is that we have done? Uh our entire aim is to ensure that we can able to do back propagation or not. Somewhere in the first uh slide we have seen that back propagation is difficult. Now, we are trying to understand why uh that uh difficult. Where is that uh difficult?

[15:40] So, we were able to uh reach till a Z very successfully. Okay? Now, with this, let us uh try to understand further. However, the difficulty is not in the uh decoder, we have understood. Now, where exactly is the problem? The real problem uh starts when uh you uh you know, you want to now pass the gradients from uh Z backwards. So, back to encoder uh parameters. So, what are encoder uh outputs here? The encoder outputs are mean and variance. Later, the variance is converted into standard uh deviation. Okay. So, the encoder outputs are this mean and uh standard deviation. Okay.

[16:23] Now, you have to compute the gradients with respect to these two. Now, let us see whether we can able to do that or not. So, during the back propagation, you have to reach until the encoder uh parameters. So, we are successful reaching till Z. From Z, now we need to update, we need to uh compute the gradients with respect to mean and variance or mean and uh standard deviation. So, we are now somewhere uh seeing that there is a problem here. So, let us uh see uh how do we pass the gradients from Z back to the encoder parameter. Now, uh but the problem uh starts when you try to update these uh parameters. Now, let us try why this uh problem has occurred.

[17:06] So, encoder is producing a mean and uh variance or mean and uh standard deviation. Okay. Once it gives this, how is that we are getting a Z? Is it not Z is a sampled from this particular distribution? Okay. Now, so we have understood that till Z you can able to back propagate it. Then encoder is giving these two parameters and a Z is a sampled from this particular distribution. Now, what is that we need to do? We have to update these encoder parameters, right? Now, in order to update these encoder parameters, we need to compute the gradients with respect to mean, we need to compute the gradients with respect to standard deviation.

[17:51] So, using the chain rule, can we write this? So, now you want to compute the log gradient of derivative of loss with respect to mean and derivative of loss with respect to the standard deviation. So, now we need these two. So, using a chain rule, can we write in this way? How do we reach from loss? How do we reach to this point during a back propagation? From loss, can I reach to Z? And from Z can we reach to this? Now, how will we reach till Z? It is already written here.

[18:29] So, using a chain rule, we can reach till that you know, Z. From Z we can reach to mean. So, this is how you reach to mean and this is how we reach it to a standard deviation. So, I hope up till this particular point it is clear. So, what is derivative of loss with respect to Z? This is the formula. Now, the problem comes here. These two are the terms. Now, you need to compute the derivative of Z with respect to mean, derivative of Z with respect to standard deviation. We want these two terms values. We want these two. Now, can we able to compute this? What is What do you think?

[19:15] What is your answer for this? Definitely no. Why it is no? Because Z is a sampled from this distribution. When it is a sampled, it is a random operation. Therefore, whatever Z equation that we have written, it cannot be directly expressed as a differentiable function of mean. The you cannot, you know, differentiate with respect to this. This is the problem. I hope this is clear to you. What exactly is an issue? We want to compute the back propagation.

[19:49] Now, in order to compute the back propagation up till a Z, it is a fine. From Z when we back propagated further to update the mean and variance, we are seeing

that there is a problem. What is the problem? Using a chain rule, we can able to reach up till this point. But we know this value, but we doesn't know these two terms. Why we doesn't know these two terms? Because Z is a sampled from this particular distribution. When Z is a sampled from this distribution, Z is a not a directly expressed as a differentiable function of mean and sigma.

[20:30] Therefore, there is a problem at this point of time. Direct back propagation through sampling is not possible. So, because of this particular you know, we require a concept called reparameterization trick. Now, let us see how if this is the case, how do we sample a Z so that we can compute the gradients. So, Z is a problem because you have sampled a Z from an distribution. If that is the problem, how do we sample a Z ? So, can we think in this way? Instead of directly sampling a Z from this particular distribution, can we express Z in this way? Can we do sampling operation? Can we write a Z in this particular form?

[21:15] Now, if you look into this from our session, the encoder is going to produce these two parameters. That is mean and then the standard deviation. Now, once it is going to give us these two, which means look at this. Is it not the here whatever are these two terms? That mainly depends on our input X and then the encoder parameters weights and bias. Agreed? Yes. Then, look at the next term. We have this uh you know uh epsilon. So, what is this epsilon? From this equation, we are seeing that epsilon is a something which is totally free from that mean and the variance. Uh mean and the standard deviation here. And epsilon value is a something which is a sampled from a normal distribution or Gaussian

[22:08] distribution. So, now you can ask the question, when the encoder is already giving us a mean and standard deviation or variance, why is that we require this epsilon uh concept? So, why do we require this epsilon? Now, do you agree to this? For a given input X , there exist a multiple such Z s. But here, if you write an equation Z in this particular form, then uh uh mean and uh variance whatever the encoder is giving, from that if you want to generate a different Z , then there has to be some randomness. That randomness is what is this epsilon giving us. Now, but at the same time, now you can ask the question, "Ma'am, again there is an randomness here." Yes, there is a randomness, but it is completely

[22:58] independent of that mean and then uh uh you know, standard deviation. So, you see here whatever the epsilon epsilon is something which is not learned by the network and its a value is being a sampled from this a particular distribution. Therefore, the network does not learn the epsilon and it is completely independent of the mean and then the standard deviation. Therefore, during a back propagation, it is a randomly picked one value, which means it is just one number. During back propagation can be assumed this to be a constant. Right?

[23:35] Yes. So, therefore, now whatever Z that we have written, Z is now differen-

table function with respect to two whatever are the two parameters of mean and then the sigma or standard deviation. This is reparameterization trick, one of the most important concept. Let me quickly summarize to you what exactly we have discussed. See, encoder is a something which is giving us the two outputs. That is mean and then the standard deviation or mean and variance. Now, from a mean and variance, when you so far what we were doing, we were sampling a Z from that distribution. Now, when we were sampling, there is an randomness here because randomly you are picking one value. Each time you pick the value

[24:23] is a different. That different value when you pass it to decoder, you get a different reconstructed one. Second time when you sample it, again you get a different. Then you get a different variant. So, but the problem is back propagation is causing an issue. Why back propagation is causing an issue? Because Z is sampled in this way. So, can we sample a Z in a slightly different way? Okay. What is that a different way? Z is a sampled from this particular distribution. That is I mean Z is a sampled using this particular equation, where you are writing a mean and then standard deviation and there is an epsilon. What is that epsilon term? Epsilon is sampled from once again a normal or a Gaussian

[25:09] distribution. Now, again there is an randomness. Yes, there is a randomness, but that randomness is completely independent of those two terms. And during back propagation, this is treated as constant. And this is a something which model need not have to learn because it is totally independent. And those two parameters, mean and standard deviation, it is depending on the input X and then the parameters, that is weights and bias. I hope this is clear to us. If it is still not clear, let us proceed further and then we will see that mathematically.

[25:51] So, what we have done? Random reparameterization trick is separating the randomness from the encoder parameter, right? Definitely it is separated. Now, this is how you are writing the equation. Now, the gradients can flow from decoder back to the encoder. How? Let us look into this. Now, can we able to the entire problem was derivative of Z with respect to mean and derivative of Z with respect to sigma or standard deviation was something which was unknown for us from the previous one. Now, why? Because Z was a sampled using this. Now, what we are doing? We have sampled a Z in a different way.

[26:39] Slightly we have by using reparameterization trick. So, now this is the Z equation. Can we now differentiate with respect to mean? Yes, let's go ahead. Derivative of Z with respect to mean here. How is that you are writing derivative this term is you are writing derivative with respect to mean and this term also derivative with respect to mean here. Now, since this is one, definitely the value is one, that's what is being written here. And in the second term, here you don't have any mean value.

[27:16] Since this term does not contain a mean, the derivative becomes zero. There-

fore, can you write this whole equation as $1 + 0$? Why? The first term is one and the second term is zero. Why second term is zero? Because you don't have the this particular doesn't have the term mean. Therefore, you now got a value. Earlier, this was the problem. We were not knowing what exactly is the value for this, but now we got the value.

[27:47] Likewise, can we now compute with respect to you know, sigma? So, when we differentiate it with respect to this term, so here you don't have this. Therefore, this term becomes zero. And here you have this. Therefore, when you take the derivative, you are going to get an epsilon value. Therefore, this particular earlier we didn't had a value, but now we are having a value for both of these terms. Right? Now, I hope this is clear to us. Earlier, the problem was we were not knowing the value of these two, but now we have a value for these two. Okay. Now, let's have a quick summary. What is that we have discussed so far?

[28:34] Decoder is one which is going to go give us the output. So, how is that it is going to give? Mathematically, you are writing the output is denoted as $\hat{X}$ and for that we are giving an input as a Z and this is an decoder neural network. Θ is the parameters and you can easily update those parameters. Okay. Then you so decoder is not an issue. From decoder we have now reached to Z . The loss depends on Z through $\hat{X}$. We have seen that. So which means how do you how is that you are writing derivative of loss with respect to Z is with respect to $\hat{X}$ and $\hat{X}$ with respect to Z .

[29:15] This is also not an issue. Now, later using a reparameterization trick Z is being sampled. How is that Z you are obtaining? Z is obtained in this way. Now because you have written in this, just some time back we have seen that now definitely Z is differentiable with respect to both of these parameters. Because of this we got this value as one and here with respect to this we got an epsilon value. Okay. So which means what was the problem earlier, we were able to solve this particular problem.

[29:51] So what is that we have done? So once the gradient reaches the latent space Z , we must now you know, the gradient has to go back to the encoder outputs. What are encoder outputs? Mean and then the standard deviation. Yes. We are able to reach to this point. How is that we are able to reach? From L from loss we are reaching to $\hat{X}$, from $\hat{X}$ to Z , from Z we are reaching to this. So is it not for mean this is the path? And how is that using a chain rule you can write? Is it not using a chain rule derivative of loss with respect to mean is loss with respect to $\hat{X}$, $\hat{X}$ with respect to Z and Z with respect to this.

[30:38] We have a value for each of these. How? This particular term value is one. So, if this value is one, then definitely derivative of loss with respect to mean, is it not this? So, is it not we can easily compute? Yes. So, similar way, this is how we will compute with respect to two a standard deviation. So, from this, how can you reach

to this particular one? This is the path. Loss from loss you will reach to $\hat{X}$. From $\hat{X}$ you will take the derivative with respect to Z , and from Z with respect to two the standard deviation. So, sometime back we have seen the value of ϵ . So, we have got the value. So, which means from loss you have reached to decoder. From decoder you have reached to Z . From Z

[31:30] you are successful reaching to the output parameters. What is only left out? You have to now go to the encoder, and in the encoder also we have weights and bias. We have to now reach there. So, what is the next step? Finally, after once the gradients has reached the encoder output, that is mean and then the standard deviation, we can now propagate to the encoder parameters. From Can you visualize this? From $\hat{X}$ we have now reached to an decoder.

[32:03] From decoder we have reached to a latent space. From latent space we have reached to these. Now we have to go to an encoder parameters. So, that is what now the gradients has to reach to an encoder parameters. We can easily do that using a standard back propagation. What is the path? First of all, you have to reach it to this parameters. Here five is an parameters of encoder, which will be weights and bias. So, how can you reach this using an chain rule?

[32:32] Mathematically, can we write derivative of loss with respect to mean? And from a mean we can reach to this particular parameter and how is that you are writing this that is this particular term? Right? So, now we can easily update the parameters. So, therefore, the reparameterization trick you know enables us to find out the gradients from the output layer and you can go back till the input. That is the encoder parameters. So, with this you can able to compute the gradients and end-to-end training is possible in VAE.

[33:13] So, this is all about the reparameterization trick. So far we have never seen the complete architecture of VAE. We had seen in bits by bits. So, this is what is the complete architecture of VAE. We have an input X that is a passed to an probabilistic encoder and encoder is a something which is going to give us this output. And how is that we are sampling a Z ? Z is being sampled by using this mean and then this term and then we are using an term ϵ . ϵ is also being added.

[33:51] And this is Z . Once you get a Z , Z is then passed to a decoder and a decoder is going to give us the output or it is $\hat{X}$ or X' . This is what is the overall architecture. And what we have discussed in this particular session is parameterization reparameterization trick, right? So, before reparameterization, Z was sampled from this particular distribution. After reparameterization, Z is obtained from this particular equation. This is the equation through which the Z is obtained.

[34:26] Because of this, Z is a differentiable with with to those parameters. With this, we have come to an end of this particular session. So, so far we have seen the detailed architecture of encoder, decoder, and then we have seen the most important

concept now called reparameterization trick. In the coming session, we will understand the entire working mechanism of VAE by solving one numerical example so that the overall flow will be very clear. With this, thank you.